

Smart Programming

```
① Class.forName("---driverClassName---");
② Connection con = DriverManager.getConnection("url", "username", "password");
③ PreparedStatement ps = con.prepareStatement('select * from register');
④ ResultSet rs = ps.executeQuery();
⑤ while(rs.next())
{
    String name = rs.getString("name");
    String email = rs.getString("email");
    // print
}
⑥ con.close();
```

name	email	password	gender	city
Sam	smith@gmail.com	123456	M	Chennai
John	john@gmail.com	654321	F	Delhi

Fetching Data from MySQL Database

Fetching data means **reading or retrieving data** from a database table.

For example:

- Show all registered users.
- Display student records.
- View employee details.

We use the **SELECT** SQL statement to fetch data.

Steps to Fetch Data from MySQL

Step 1: Load the Driver Class

Before connecting to MySQL, Java needs to load the MySQL JDBC Driver.

Code

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

Explanation

- `Class.forName()` loads the JDBC Driver.
- `"com.mysql.cj.jdbc.Driver"` is the MySQL Driver class.
- Every database has its own driver.
 - MySQL → `com.mysql.cj.jdbc.Driver`
 - Oracle → Different Driver
 - PostgreSQL → Different Driver

Note: Driver class name depends on the database you are using.

Step 2: Add the MySQL Connector JAR File

NOTE: Perform this step only when you are creating a new project.

Java cannot communicate with MySQL without the JDBC Driver.

So, you must add the MySQL Connector JAR file to your project.

In Eclipse

Project



Build Path



Add External JARs

Select the MySQL Connector JAR file.

Step 3: Create the Database Connection

Now connect Java with the MySQL database.

Code

```
Connection con = DriverManager.getConnection(
    "jdbc:mysql://localhost:3306/jdbc_db",
    "root",
    "root"
);
```

Explanation

Here,

- `localhost` → Database is running on your computer.
- `3306` → MySQL default port number.
- `jdbc_db` → Database name.
- `"root"` → Username.
- `"root"` → Password.

If everything is correct, the connection is created successfully.

Step 4: Write the SQL SELECT Query

To fetch data, use the **SELECT** statement.

Example

```
SELECT * FROM register;
```

Explanation

- `SELECT` → Read data.
 - `*` → Fetch all columns.
 - `register` → Table name.
-

Step 5: Prepare the SQL Statement

Send the SQL query to MySQL.

Code

```
PreparedStatement ps = con.prepareStatement(query);
```

Explanation

- `PreparedStatement` prepares the SQL query.
 - It is safer and faster than using a normal `Statement`.
-

Step 6: Execute the Query

This step is different from Insert, Update, and Delete.

For Insert/Update/Delete

```
executeUpdate();
```

For Fetching Data

```
ResultSet rs = ps.executeQuery();
```

Explanation

- `executeQuery()` executes the SELECT query.
 - It returns a **ResultSet** object.
 - The **ResultSet** contains all the rows returned by the query.
-

Step 7: Read Data Using ResultSet

Use a `while` loop to read each row.

Code

```
while(rs.next())  
  
{  
  
    // Read data here  
  
}
```

Explanation

`rs.next()` moves the cursor to the next row.

It returns:

- `true` → Next row exists.
- `false` → No more rows.

The loop continues until all rows are read.

Step 8: Get Column Values

Inside the `while` loop, retrieve the values of each column.

Example

```
String name = rs.getString("name");
```

```
String email = rs.getString("email");
```

Explanation

`getString("column_name")` returns the value stored in that column.

Important: The column name must exactly match the column name in the database.

Example column names:

name

email

password

gender

city

Step 9: Print or Use the Data

After retrieving the values, print them or use them in your program.

Example

```
System.out.println(name);
```

```
System.out.println(email);
```

Or print all values together:

```
System.out.println(name + " " + email);
```

You can also display the data in a table, GUI, or webpage.

Step 10: Close the Connection

After all data has been read, close the database connection.

Code

```
con.close();
```

Why?

Closing the connection:

- Frees system resources.
 - Prevents memory leaks.
 - Improves application performance.
-

Difference Between executeUpdate() and executeQuery()

executeUpdate()

Used for INSERT

Used for UPDATE

executeQuery()

Used for SELECT

Fetches data

Used for DELETE

Returns ResultSet

Returns an integer (number of affected rows)

Returns ResultSet object

What is ResultSet?

A **ResultSet** is an object that stores the data returned by a **SELECT** query.

Think of it like a table in memory.

Example:

Name	Email	City
Amit	amit@gmail.com	Delhi
Rahul	rahul@gmail.com	Mumbai
Priya	priya@gmail.com	Pune

The `ResultSet` stores all these rows, and `rs.next()` moves through them one by one.

Complete Flow of Fetching Data

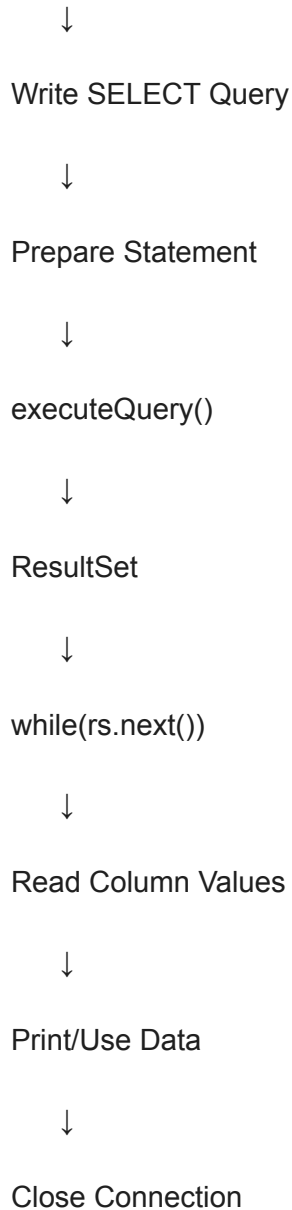
Load Driver



Add MySQL JAR



Create Connection



Important Points to Remember

- Use **SELECT** to fetch data.
- Use `executeQuery()` instead of `executeUpdate()`.
- `executeQuery()` returns a **ResultSet**.
- Use `rs.next()` to move through each row.

- Retrieve values using methods like `getString("column_name")`.
 - Column names must exactly match those in the database.
 - Always close the database connection after finishing.
-

Quick Revision

1. Load the JDBC Driver.
2. Add the MySQL Connector JAR.
3. Create the database connection.
4. Write the `SELECT` query.
5. Prepare the statement.
6. Execute the query using `executeQuery()`.
7. Store the result in a `ResultSet`.
8. Use `while(rs.next())` to read each row.
9. Retrieve column values using `getString()`, `getInt()`, etc.
10. Print or use the data.
11. Close the database connection.

Easy Memory Trick:

Driver → **JAR** → **Connection** → **SELECT** → **Prepare** → **executeQuery** →
ResultSet → **next()** → **getData** → **Print** → **Close**

Vide Link:  [#4 Fetch Data from MySQL Database using JDBC in Java || ResultSet Interfac...](#)